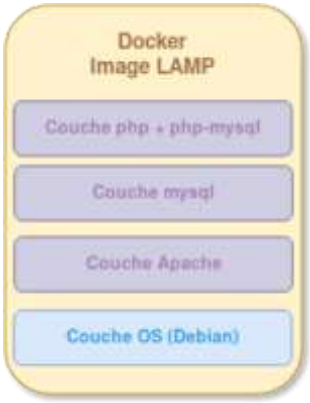
Année universitaire : 2025/2026		Classes : GLSI-3
Module : Virtualisation et Cloud TP4 : DockerFile		

Nous allons créer notre propre stack LAMP (Linux Apache MySQL PHP) au moyen de Docker.Voici les différentes couches de cette image :

- Une couche OS pour exécuter notre Apache, MySQL et Php, je vais me baser sur ladistribution Debian.
- Une couche Apache pour démarrer notre serveur web.
- Une couche php qui contiendra un interpréteur Php mais aussi les bibliothèques quivont avec.
- Une couche Mysql qui contiendra notre système de gestion de bases de données.

Voici le schéma de notre image :



1 - Création des sources et du Dockerfile

Commencez par créer un dossier et téléchargez les sources de l'image :

$ mkdir LAMP puis $ cd LAMP

$ wget https://devopssec.fr/documents/docker/dockerfile/sources.zip

$ unzip sources.zip

Ensuite dans la racine du dossier que vous venez de créer, créez un fichier et nommezle **Dockerfile**.
Voici l'arborescence que vous êtes censé avoir :

```
├── app
│   ├── db-config.php
│   └── index.php
├── db
│   └── articles.sql
└── Dockerfile
```

- *db* : contient un fichier *articles.sq* , qui renferme toute l'architecture de la base dedonnées.
- *app* : comporte les sources php de notre

application web

- Dans le fichier **Dockerfile**, rajoutez le contenu suivant

```
# Base OS
FROM debian:stable-slim

LABEL version="1.0" maintainer="GI3"

ARG APT_FLAGS="-q -y"
ARG DOCUMENTROOT="/var/www/html"

# Apache
RUN apt-get update && \
    apt-get install $APT_FLAGS apache2

# MariaDB
RUN apt-get install $APT_FLAGS mariadb-server

COPY db/articles.sql /

# PHP
RUN apt-get install $APT_FLAGS php php-mysql && \
    rm -f $DOCUMENTROOT/index.html && \
    apt-get autoclean -y

COPY app $DOCUMENTROOT

EXPOSE 80
WORKDIR $DOCUMENTROOT

ENTRYPOINT ["bash", "-c", "mysqld_safe & sleep 5 && mysql < /articles.sql
&& apache2ctl -D FOREGROUND"]
```

Remarque :
il faut savoir qu'un conteneur **se ferme automatiquement à la fin de son processus principal**. Il faut donc un processus qui tourne en premier plan pour que le conteneur soittoujours à l'état running, d'où le lancement du service Apache en premier plan à l'aide de lacommande **apache2 -D FOREGROUND**.

2- Construction et Exécution de notre image

Voici la commande pour qui nous permet de construire une image docker depuis un Dockerfile :

$ docker build -t <IMAGE_NAME> .

Ce qui nous donnera :

$ docker build -t my_lamp .

Ensuite, exécutez votre image personnalisée :

$ docker run -d --name my_lamp_c -p 8080:80 my_lamp

Visitez ensuite la page suivante http://localhost:8080/

3 - Publier son image dans le Hub Docker

a - Créer un repository public dans docker hub

b - L'étape suivante est de se connecter au hub Docker à partir de la ligne de commande

$ docker login

Il va vous demander, votre nom d'utilisateur et votre mot de passe, et si tout se passe bienvous devez avoir le message suivant : Login Succeeded

c- Récupérer ensuite l'id ou le nom de votre image :
$ docker images ls

d - Ensuite il faut rajouter un tag à l'id ou le nom de l'image récupérée. Il existe une commande pour ça :
$ docker tag my_lamp account/lamp:first

E - Maintenant pousser votre image vers le Hub Docker grâce à la commande suivante :
$docker push account/lamp:first

Amélioration de notre image LAMP avec le concept de Volume

L'image avec stack LAMP, malheureusement, ce n'est pas top niveau persistance de données car, lors d'un redémarrage du conteneur, nous allons rencontrer les deux problèmessuivants

- Les données de notre base ne sont pas sauvegardées.
- Les modifications des sources de notre application ne seront pas appliquées.

Pour résoudre ce problème, nous allons utiliser les volumes ! Créer un nouveau dossier et Commencez par télécharger le projet :
$ wget https://devopssec.fr/documents/docker/volumes/project.zip

$ unzip project.zip

$ cd project

Dans ce projet, il s'agit du même Dockerfile, mais juste on a changé les fichiers sources enrajoutant un formulaire.
Dans le fichier Dockerfile, mettre cette instruction en mode commentaire :
COPY app ${DOCUMENTROOT}

Créer votre image :

$ docker build -t my_lamp2 .

Concernant la base de données, nous allons créer un volume nommé "mysqldata" :

$ docker volume create mysqldata

Lister les volumes :

$ docker volume ls

Nous allons aussi changer le dossier source en utilisant le concept de volume. Lançons doncnotre conteneur :

$ docker run -d --name my_lamp2_c -v $PWD/app:/var/www/html -vmysqldata:/var/lib/mysql -p 8080:80 my_lamp2

La commande $PWD prendra automatiquement le chemin absolu du dossier courant, doncfaites bien attention à exécuter votre image depuis le dossier du projet où mettez le chemincomplet si vous souhaitez lancer votre commande depuis n'importe quelle autre chemin.

Remarque :

Supprimer tous les volumes locaux non
inutilisésdocker volume prune
Supprimer un conteneur Docker avec le/les volumes
associésdocker rm -v <CONTAINER_ID ou
CONTAINER_NAME>
-v ou --volume : supprime les volumes associés au conteneur

Rappel pour les instructions DockerFile les plus communément utilisées.

- **FROM**: Définit l'image de base qui sera utilisée par les instructions suivantes.
- **LABEL**: Ajoute des métadonnées à l'image avec un système de clés-valeurs, permet par exemple d'indiquer à l'utilisateur l'auteur du Dockerfile.
- **ARG** : Variables temporaires qu'on peut utiliser dans un Dockerfile.
- **ENV** : Variables d'environnements utilisables dans votre Dockerfile et conteneur.
- **RUN**: Exécute des commandes Linux ou Windows lors de la création de l'image. Chaque instruction **RUN**va créer une couche en cache qui seraréutilisée dans le cas de modification ultérieure du Dockerfile.
- **COPY**: Permet de copier des fichiers depuis notre machine locale vers le conteneur Docker.
- **ADD**: Même chose que COPY mais prend en charge des liens ou des archives(si le format est reconnu, alors il sera décompressé à la volée).
- **ENTRYPOINT** : comme son nom l'indique, c'est le point d'entrée de votre conteneur, en d'autres termes, c'est la commande qui sera toujours exécutéeau démarrage du conteneur. Il prend la forme de tableau JSON (ex : CMD ["cmd1","cmd1"]) ou de texte.
- **CMD**: Spécifie les arguments qui seront envoyés au **ENTRYPOINT**, (on peut aussi l'utiliser pour lancer des commandes par défaut lors du démarrage d'un conteneur). Si il est utilisé pour fournir des arguments par défaut pour l'instruction **ENTRYPOINT**, alors les instructions **CMD**et **ENTRYPOINT**doivent être spécifiées au format de tableau JSON.
- **WORKDIR**: Définit le répertoire de travail qui sera utilisé pour le lancement des commandes **CMD**et/ou **ENTRYPOINT**et ça sera aussi le dossier courant lors du démarrage du conteneur.

- **EXPOSE**: Expose un port.
- **VOLUMES**: Crée un point de montage qui permettra de persister les données.
- **USER** : Désigne quel est l'utilisateur qui lancera les prochaines instructions **RUN**, **CMD**ou **ENTRYPOINT**(par défaut c'est l'utilisateur root).